Talking Paper

Event-Driven Order Fulfillment System

System Integration · Spring 2026

Full spoken script for each of the 11 slides. Exam format: individual external exam — presentation ~10 min, discussion ~15 min, total 30 min (Henrik Kuhl, SYS Spring 2026).

Time Budget

#	Slide	Target	Running
1	Title	0:20	0:20
2	Why Event-Driven?	1:00	1:20
3	System Architecture	1:15	2:35
4	Sync vs. Async	1:00	3:35
5	Choreography Saga	1:20	4:55
6	Idempotency	1:00	5:55
7	Retries & DLQ	0:55	6:50
8	Fan-Out / Pub-Sub	0:50	7:40
9	Testing Strategy	1:05	8:45
10	Docker & Deployment	0:50	9:35
11	Key Takeaways	0:30	10:05

SLIDE 1 OF 11

Event-Driven Order Fulfillment System

Key points:

```
Target: ~20 sec
```

What to say:

Good morning. My name is Marco and I will present my synopsis project: an Event-Driven Order Fulfillment System in .NET 10. I will walk through the architecture, four design patterns, and the testing strategy in about ten minutes.

SLIDE 2 OF 11

Why Event-Driven?

Key points:

```
Target: ~60 sec
POST /orders -> publishes OrderPlaced -> HTTP 201 immediately
One failure does NOT cascade
```

What to say:

Synchronously, every service call blocks the next — the customer waits for the sum of all latencies, and if Shipping goes down the entire chain fails. Event-driven: the Order API publishes one event and returns HTTP 201 immediately. RabbitMQ routes a copy to every subscriber in parallel. Services are decoupled in time and space — one failure no longer cascades.

SLIDE 3 OF 11

System Architecture

Key points:

```
Target: ~75 sec
7 services + RabbitMQ + PostgreSQL + StubPaymentGateway
Shared Contracts library — compile-time type safety
Failure path: PaymentFailed -> StockReleased + OrderCancelled
```

What to say:

Seven microservices, one shared Contracts library, RabbitMQ as the broker, and PostgreSQL for state. The horizontal chain is the core saga: OrderService publishes OrderPlaced, StockConsumer reserves stock, PaymentConsumer charges the card, then OrderService publishes OrderConfirmed. That single event fans out via a RabbitMQ fanout exchange to four consumers in parallel — Invoice, Warehouse, Shipping, Notification — each with its own named queue. Failure path: if payment fails, compensating events undo earlier steps.

SLIDE 4 OF 11

Synchronous vs. Asynchronous

Key points:

```
Target: ~60 sec
Sync: tight coupling, cascading failure, 720 ms wait
Async: loose coupling, independent failure, 48 ms ACK
Rule: async when result not needed immediately
```

What to say:

The table shows the key trade-offs. Synchronous gives strong consistency and a simple call stack — but tight coupling and cascading failures. Async gives loose coupling and immediate responses — but eventual consistency and distributed debugging. The bar chart: 720 ms versus 48 ms to the client. The rule: choose async when the caller does NOT need the result right now.

SLIDE 5 OF 11

Choreography Saga Pattern

Key points:

```
Target: ~80 sec
No 2PC — each service commits locally
PaymentFailed -> StockReleased + OrderCancelled (compensation)
Choreography: no central controller
```

What to say:

The Saga pattern replaces a distributed transaction with local commits — no 2-Phase Commit, no distributed locks. Each service commits locally and publishes the next event. Top row: the happy path. Bottom row: payment fails — StockConsumer hears PaymentFailed and publishes StockReleased, releasing the reserved inventory. OrderService publishes OrderCancelled. These are compensating transactions — forward-only, not rollback. We use choreography: no central controller; each service knows only its own role.

SLIDE 6 OF 11

Idempotency — Duplicate Messages

Key points:

```
Target: ~60 sec
Unique constraint on ProcessedMessages.MessageId
DbUpdateException -> catch -> return (ack, not requeued)
```

What to say:

RabbitMQ guarantees at-least-once delivery — not exactly-once. Every consumer must be idempotent. Our mechanism: before any business logic, we INSERT the MessageId into a ProcessedMessages table with a unique constraint. If the row already exists, DbUpdateException is thrown — we log a warning and return without processing. The message is acknowledged so it is not requeued. First delivery processes; every retry is silently discarded.

SLIDE 7 OF 11

Resilience — Retries & Dead-Letter Queues

Key points:

```
Target: ~55 sec
e.UseMessageRetry(r => r.Interval(3, TimeSpan.FromSeconds(2)))
DLQ: shipping-service_error — visible in RabbitMQ UI at :15672
```

What to say:

Transient failures — a network blip, a database timeout — are normal. Our mechanism: MassTransit retries up to three times with two seconds between attempts. If all three fail, the message moves to shipping-service_error — a dead-letter queue visible in the RabbitMQ UI on port 15672. An operator can inspect the message, fix the root cause, and requeue it. No message is ever silently lost.

SLIDE 8 OF 11

Fan-Out — Publish-Subscribe

Key points:

```
Target: ~50 sec
Fanout exchange -> one message, N independent queues
Adding a subscriber = zero changes to the publisher
```

What to say:

OrderService publishes OrderConfirmed to a RabbitMQ fanout exchange. RabbitMQ delivers a copy to every bound queue — one per consumer. All four run in parallel, completely independently. The key property: the publisher does not know who is listening. Adding a new consumer means creating a new service and binding its queue — zero changes to OrderService. Slow Invoice processing does not block Shipping.

SLIDE 9 OF 11

Testing Strategy

Key points:

```
Target: ~65 sec
xUnit: [Fact] tests + assertions
Moq: mock ConsumeContext, verify Publish() Times.Once
WireMock: real HTTP stub POST /charge -> 200 or 402
Testcontainers: real Postgres, unique constraint verification
```

What to say:

Four tools, four layers. xUnit runs the tests. Moq mocks ConsumeContext so we test consumer logic without a live broker — we verify that Publish was called exactly once with the correct payload. WireMock.Net starts a real HTTP server to test PaymentGatewayClient — we stub POST /charge to return 200 or 402. Testcontainers starts a real PostgreSQL 16 container to verify the unique constraint — an InMemory database cannot enforce that constraint.

SLIDE 10 OF 11

Docker & Deployment

Key points:

```
Target: ~50 sec
docker compose up --build — one command, nine containers
Multi-stage Dockerfile: SDK build stage + runtime stage
depends_on: condition: service_healthy
```

What to say:

One command — docker compose up --build — starts all nine containers. Multi-stage Dockerfiles keep images small: SDK image for build, runtime image for execution. Health checks on RabbitMQ and PostgreSQL prevent race conditions where a consumer starts before the broker is ready. Configuration is injected via environment variables, so the same images run locally and in production.

SLIDE 11 OF 11

Key Takeaways

Key points:

```
Target: ~30 sec
Benefits: loose coupling, resilience, independent scalability
Trade-offs: eventual consistency, distributed debugging, idempotency
required
```

What to say:

To summarise: four patterns, seven services, ten tests, nine containers. Event-driven gives loose coupling, resilience, and independent scalability. The trade-offs are real: eventual consistency, distributed debugging, and mandatory idempotency. I am happy to discuss any of this.

Appendix: Curriculum Topics

EasyNetQ vs MassTransit

EasyNetQ is a simple RabbitMQ client — quick to start but requires manual retry and DLQ implementation. MassTransit provides built-in abstractions for all of these.

API Gateway (Ocelot)

An API gateway provides a single entry point. Ocelot is configured via JSON — routing, authentication, rate limiting, and response aggregation.

Security — HashiCorp Vault

Instead of hardcoding credentials in environment variables, a production system fetches secrets from Vault at startup. Vault provides dynamic secrets and audit logs.

Event Sourcing (differs from event-driven)

This project is event-driven but NOT event-sourced. Event Sourcing stores every state change as an immutable event; current state is derived by replaying events.

Sidecar / BFF / Anti-Corruption Layer

Sidecar: helper container alongside the main service. BFF: dedicated API layer per client type. ACL: translates between bounded contexts with different domain models.

Sam Newman — Building Microservices

Key themes: database-per-service (we follow this), bounded contexts, and the danger of distributed monoliths. Our system communicates only through events.